Overview

CSE 291A: Systems for LLMs and AI Agents

Fall 2026 TENTATIVE Course Projects

The course project is an opportunity to investigate a systems question about language models, AI agents, or AI-assisted security. Choose one of the seven options below. Each project has two phases: first establish a reproducible baseline and measure its behavior; then develop and evaluate an improvement or conduct a deeper controlled study.

The emphasis is on a clear research question, sound system design, careful measurement, and evidence that explains the results. A strong project may demonstrate a useful improvement, identify when an approach fails, or establish that a simpler baseline is sufficient. Unsupported claims of better performance or security are not a substitute for a controlled experiment.

Project options

1. Memory-Aware Agent Serving
2. CPU/GPU/NPU-Aware Agent Pipelines
3. Adaptive Routing Across Heterogeneous Edge Machines
4. Security–Performance Tradeoffs in Edge Agents
5. Failure Propagation and Recovery in Multi-Agent Systems
6. Budget-Aware AI-Assisted Fuzzing
7. Resource-Bounded Cybersecurity Challenge Agents

Organization and milestones

Projects may be completed individually or in groups of 2–4 students (**1–4 students per group**). Phase 1 implementation and measurements, and the Phase 1 writeup, are due **Wednesday, October 28, 2026**. Phase 2 implementation and evaluation, and the final project report, are due **Thursday, December 10, 2026**.

Phase 1 should produce a working baseline, a reproducible workload and evaluation procedure, initial measurements, and a justified Phase 2 direction. Phase 2 should produce the completed system or measurement study, comparisons against appropriate baselines, an explanation of the results, and reproducible artifacts.

For both phases, submit the relevant source code, dataset, experiment instructions, results and logs needed to support the claims, and a concise report. The final report should describe the

question, design, experimental setup, results, and limitations. Individual contribution statements or breakdowns of who did what are not required. If submitted, they will not be considered in grading.

This is a group project: **every student in a group receives the same score** on each project component and the same overall project score.

Computing resources

Two shared edge machines are available for potential project use: a GMKtec EVO-X2 with AMD Ryzen AI Max+ 395, and an ASUS NUC 16 Pro, model RNUC16GDKU76000U. Installed RAM, storage, operating systems, supported accelerator runtimes, and access procedures will be confirmed before experiments are assigned. Do not assume that a particular model or operator runs on an NPU merely because the machine has one.

Each student has $50 in university-provided AWS credits. Eligible services and access restrictions remain to be confirmed; projects must not depend on cloud GPU instances, paid model APIs, or pooling credits. Laptop development, CPU-based experiments, and bounded access to shared local inference should support a viable core project.

Shared-machine experiments will need coordinated access. Record concurrent activity and reserve uncontended periods for final performance measurements. Develop workload generators and analysis locally where possible, then validate the full system on the intended hardware.

Evaluation and scoring

Assessment will consider baseline reproduction and measurement quality; the justification and implementation of the Phase 2 design; experimental controls and comparison quality; depth of analysis; and reproducibility and clarity of the report.

Use development workloads to design the method and separate evaluation workloads to assess it. Pin relevant model, runtime, dataset, and code versions. Account for setup and compilation when they affect the use case, and distinguish cold-start from steady-state behavior. Record failed runs and timeouts rather than silently excluding them. Repeat stochastic experiments and report variability.

Security outcomes should be supported by programmatic checks, execution traces, or inspected evidence. Distinguish a model’s inability to complete a task from successful protection. Use course-provided or approved local benchmarks and simulated resources. New vulnerabilities, model training, and publication are not requirements.

Option 1: Memory-Aware Agent Serving

Option 1: Memory-Aware Agent Edge Serving

Project description

Agent workloads generate multiple dependent model requests, use tools between requests, and retain context across steps. On a memory-constrained machine, admitting too much work can increase latency or exhaust memory; admitting too little can leave resources idle. This project investigates how a serving system should manage concurrent agent tasks and their context under a limited memory budget.

The core question: How can admission and scheduling decisions improve completed agent work while keeping memory use bounded?

Hardware and software environment

Use one shared edge machine with a supported local inference runtime, an agent workload runner, and a queue or scheduler under your control. Keep the model and precision fixed in the main comparisons. Begin with workloads that fit comfortably on the selected machine, then vary concurrency and context size. Any cache-aware extension must use cache controls and measurements that the runtime actually exposes.

Phase 1: Baseline implementation and measurement

Implement a FIFO queue with fixed concurrency. Create a mixture of short and long contexts, short and long responses, and multi-step tasks with controlled tool delays. Instrument both individual model requests and complete agent tasks.

Measure task completion time, queueing time, tail latency, throughput, peak memory, and failures or timeouts. Determine how concurrency, context growth, and idle periods between agent steps affect memory pressure and completed work. Characterize whether retained contexts or reusable prefixes materially affect the chosen runtime.

Deliver a reproducible baseline, workload definitions, performance curves, and an explanation of the dominant bottleneck.

Phase 2: Memory-aware scheduling

Build a policy motivated by Phase 1. Possible mechanisms include estimating the memory needed by incoming requests, adapting concurrency to observed pressure, prioritizing ready agent steps, or choosing which reusable context to retain when supported by the runtime.

Select a focused mechanism and compare it against FIFO and the best fixed-concurrency baseline selected on development workloads. A cache-aware design should also compare against a simple fixed cache policy. Evaluate changing arrival rates and workload mixes, including cases where the policy provides little benefit.

Evaluation and deliverables

Report completed tasks, median and tail completion latency, throughput, peak memory, and failure rates. Check that performance gains do not come from skipping tasks, shortening outputs, or reducing task quality. Account for policy overhead and test whether long tasks are unfairly delayed.

Submit the scheduler, workload runner, raw measurements, analysis scripts, and a report connecting observed behavior to memory-management decisions.

Detailed assessment criteria

Strong submissions explain the memory bottleneck, justify the policy from measurements, compare against tuned simple baselines, and demonstrate reproducible end-to-end behavior. Implementing many policies is less valuable than understanding one well.

Starting resources

- llama.cpp
- AMD ROCm compatibility documentation

Option 2: CPU/GPU/NPU-Aware Agent Pipelines

Option 2: CPU/GPU/NPU-Aware Agent Pipelines

Project description

An agent application can combine generation, embedding, reranking, and perception. These stages differ in model size, input shape, data movement, and runtime support. This project studies how to place stages on the CPU, GPU, or NPU of an edge machine.

The core question: Which execution device should each stage use, and when should that choice change with workload or load?

Hardware and software environment

Use a supported runtime on either shared edge machine. CPU and GPU execution form a viable baseline; include NPU execution where the selected models and software support it. Document unsupported operations and CPU fallbacks. A project should not depend on implementing missing NPU operators or compiler support.

Phase 1: Measurement and profiling

Choose a small pipeline with at least two meaningful stages, such as retrieval with embedding/reranking followed by generation. Profile supported stages on the available devices. Vary relevant inputs: prompt and output length, batch size, candidate count, or concurrent requests.

Measure per-stage and end-to-end latency, throughput, memory, initialization and compilation time, and data-transfer overhead. Separate warm execution from cold starts. Measure energy only if reliable instrumentation is available, and state whether measurements cover the accelerator, processor package, or whole machine.

Determine actual device placement and identify runtime fallbacks. Produce a support matrix and performance profiles showing which device is preferable under which conditions.

Phase 2: Profile-driven placement

Use the measured profiles to build a stage-placement policy. It may consider input size, estimated stage duration, device queue state, transfer costs, and model residency. A lightweight lookup table or performance model is sufficient if it captures the important behavior.

Compare against fixed placements and the best static configuration selected on development workloads. Evaluate on new workload mixes and concurrent load. Include switching and transfer overhead rather than measuring only isolated kernel or model execution.

Evaluation and deliverables

Report end-to-end latency, throughput, memory use, policy overhead, and output quality. Keep model and precision matched where possible; separate comparisons that change the model or numerical format. Include negative results when a device or dynamic placement provides no advantage.

Submit the profiling harness, measured device profiles, placement policy, pipeline implementation, and analysis.

Detailed assessment criteria

Strong submissions verify where computation actually runs, identify the causes of device differences, and show whether placement improves complete application behavior. Unsupported NPU execution is a documented experimental finding, not grounds for inventing performance results.

Starting resources

- OpenVINO GenAI inference on NPU
- llama.cpp
- AMD ROCm compatibility documentation

Option 3: Heterogeneous Edge Routing

Option 3: Adaptive Routing Across Heterogeneous Edge Machines

Project description

The two edge machines differ in processor architecture, graphics capability, memory system, and software support. These differences may change request completion time, but they do not automatically imply that a complex router is useful. This project first establishes the differences experimentally, then uses the findings to design request routing.

The core question: When should a request run on each machine, considering workload characteristics, queueing, and communication overhead?

Hardware and software environment

Use both shared machines as inference endpoints and a lightweight routing service. Use a wired connection where possible and measure communication overhead. Route complete inference requests or agent steps; splitting one model across machines is outside the core project.

Phase 1: Comparative measurement

Run matched model checkpoints and quantization settings on both machines wherever possible. Vary model size, prompt/context length, response length, request concurrency, and workload mix. Measure prefill and decode behavior, end-to-end latency, throughput, memory consumption, and queueing.

Separately evaluate each machine's best practical configuration, clearly distinguishing hardware comparisons from comparisons that change models or precision. Measure network and request-transfer overhead, including the cost of sending accumulated context.

Identify whether the preferred endpoint changes with request characteristics or current load. Determine whether one machine consistently dominates in an unloaded setting and whether the other becomes useful under contention. These findings, rather than assumed hardware roles, should determine Phase 2.

Phase 2: Measurement-driven routing

Build a router that estimates completion time using the Phase 1 profiles and current queue state. Compare against always routing to each individual machine, round-robin, and a simple load-aware baseline. Evaluate steady and bursty arrivals and held-out workload mixtures.

Using different models on the endpoints is an optional extension. If chosen, measure task quality and enforce an explicit quality objective; do not attribute gains from a smaller model solely to hardware routing. Avoid moving an active session when the transfer or lost cache reuse outweighs the expected benefit.

Evaluation and deliverables

Measure throughput, tail latency, deadline attainment, task quality, network traffic, and routing overhead. Report performance when the faster endpoint is idle as well as busy. A well-supported conclusion that a simple policy is sufficient is a valid result.

Submit the comparative profiling harness, endpoint services, router, traces, and analysis explaining when routing helps and why.

Detailed assessment criteria

Strong submissions control hardware/model confounders, derive the policy from measured differences, use meaningful simple baselines, and account for complete request costs.

Starting resources

- GMKtec EVO-X2 product information
- ASUS NUC 16 Pro specifications
- llama.cpp

Option 4: Edge Security–Performance Tradeoffs

Option 4: Security–Performance Tradeoffs in Edge Agents

Project description

Edge deployments often use quantization and restricted context to reduce memory and inference cost. This project examines how those choices affect legitimate task completion and resistance to prompt injection in a tool-using agent.

The core question: Which deployment configurations provide useful performance while retaining task capability and resisting malicious instructions in untrusted content?

Hardware and software environment

Use small local models on either edge machine and CPU-based simulated tools. Begin with a manageable subset of an existing agent-security benchmark. Select models that can complete enough clean tasks for security comparisons to be meaningful. No paid model API or cloud GPU is required for the core experiment.

Phase 1: Baseline benchmark and measurements

Construct paired clean and adversarial tasks. The adversarial version introduces malicious instructions through retrieved content or tool output while preserving the legitimate user objective. Adapt a small subset of AgentDojo or an equivalent reproducible environment.

Establish task completion, unauthorized actions, latency, token consumption, and peak memory for the baseline configuration. Inspect failures and classify task misunderstanding, invalid tool calls, attack following, and unnecessary refusal. Prefer observable actions and deterministic checks over an unsupported model-generated safety score.

Report attack success across all evaluated cases and separately on tasks the configuration can complete cleanly. Do not count general inability to use tools as successful resistance.

Phase 2: Controlled deployment study

Study two factors in depth: weight quantization and context management. Compare supported precision levels of the same model checkpoint, and compare full context, truncation, and summarization within a defined context budget.

Hold the agent harness, tool permissions, task set, and scoring fixed. Record exactly which content each context policy retains; distinguish removal of the attack itself from improved resistance when the attack is still visible. Use fixed seeds or repeated runs as appropriate, and evaluate held-out paired tasks.

The main contribution is a careful benchmark and explanation of the resulting tradeoffs. A new defense is optional.

Evaluation and deliverables

Report legitimate task success, attack success, false refusals, latency, token cost, and memory. Analyze representative failures, interactions between factors, and configurations that are dominated by alternatives. Do not assume that lower precision must reduce security; a well-supported finding of little effect is valuable.

Submit the evaluation harness, paired tasks, configuration files, action traces, results, and deployment recommendations supported by the data.

Detailed assessment criteria

Strong submissions separate security from capability, control experimental confounders, use valid outcome checks, and explain security/performance tradeoffs rather than merely ranking models.

Starting resources

- AgentDojo
- Investigating the Impact of Quantization Methods on the Safety and Reliability of Large Language Models

Option 5: Multi-Agent Failure and Recovery

Option 5: Failure Propagation and Recovery in Multi-Agent Systems

Project description

Multi-agent systems pass intermediate results through messages, delegated tasks, and shared state. An incorrect or malicious result from one agent may influence later decisions by others. This project studies how such failures propagate and how a system can recover without repeating all completed work.

The core question: Can tracking dependencies between agent results support efficient recovery while preserving correct and safe task execution?

For example, a researcher provides a fabricated fact to a planner, and an executor acts on the resulting plan. Once the researcher result is identified as faulty, the system must determine which later outputs remain usable and which need to be recomputed.

Hardware and software environment

Build a small workflow, such as researcher–planner–executor, with programmatically checkable outcomes. Multiple logical agents may share one model endpoint; separate GPUs are unnecessary. Use CPU-based orchestration and simulated tools with reversible state changes.

Phase 1: Fault injection and propagation measurement

Implement and measure a clean baseline workflow. Inject faults at controlled stages: incorrect intermediate results, malicious instructions, timeouts, or stale outputs. Keep accidental faults and adversarial cases separate in the analysis.

Record messages, intermediate artifacts, tool actions, and dependencies. Measure clean task success, failure propagation, unauthorized actions, affected stages, wasted work, and end-to-end latency. Determine whether the workload actually depends on the injected result; an ignored message is not evidence of robust recovery.

Establish full restart as a recovery baseline. Include always verifying intermediate results as a comparison where meaningful, accounting for its extra inference cost.

Phase 2: Selective recovery

Implement one focused recovery mechanism. A suitable default is dependency-aware invalidation: tag intermediate results with their sources, identify downstream results that depend on a faulty output, restore affected simulated state, and rerun only the necessary steps.

Initially supply the identity of the faulty result to every recovery method so that recovery quality is evaluated independently of fault detection. Automated detection is an optional extension and must report false positives and missed faults separately.

Compare against full restart and the verification baseline under identical fault schedules and resource budgets. Evaluate faults at different stages and cases involving shared state. Report actions that cannot be undone rather than counting recomputation as complete recovery.

Evaluation and deliverables

Measure recovered task correctness, unauthorized actions, number of recomputed stages, retained valid work, recovery latency, inference cost, and tracking overhead. Check that selectively reused results really remain valid.

Submit the workflow, injection harness, dependency/recovery mechanism, traces, and an analysis of where selective recovery succeeds or fails.

Detailed assessment criteria

Strong submissions distinguish detection from recovery, demonstrate meaningful propagation paths, handle shared-state dependencies, and explain the tradeoff between recovery precision and bookkeeping overhead. No general guarantee against arbitrary malicious agents is expected.

Starting resources

- TAMAS: Threats and Attacks in Multi-Agent Systems
- A2ASecBench

Option 6: Budget-Aware AI-Assisted Fuzzing

Option 6: Budget-Aware AI-Assisted Fuzzing

Project description

Coverage-guided fuzzers repeatedly execute a program with varied inputs to explore code paths and find bugs. A fuzzing harness adapts generated inputs to a library's API. AI-generated harnesses may expose new functionality, but generation, compilation, repair, and invalid attempts all consume resources.

The core question: When should a fuzzing system spend its limited budget on AI assistance instead of continuing conventional fuzzing?

Hardware and software environment

Use a small set of C/C++ libraries with pinned, validated build environments and working baseline harnesses. Run compilation and fuzzing on CPU workers with appropriate sanitizers and coverage instrumentation. Use a local model endpoint for intermittent harness generation and repair; the core project does not require model training or production OSS-Fuzz deployment.

Phase 1: Baseline implementation and profiling

Start with two or three small libraries and a few selected APIs. Measure conventional fuzzing with supplied harnesses, then add one-shot LLM-generated harnesses.

Profile harness generation, compilation, repair attempts, and fuzzing. Measure compilability, valid API usage, target-library coverage, execution speed, and resource consumption. Plot coverage growth over time and identify where progress stalls.

Check runtime failures to distinguish library bugs from incorrect harness setup, invalid API use, and harness bugs. Save reproducible configurations, initial corpora, random seeds, and logs.

Phase 2: Adaptive budget allocation

Build a controller that chooses among continuing a productive fuzzing run, generating a harness for another API, repairing a failed harness, and abandoning an unproductive candidate. Decisions should use measured feedback such as recent coverage gain, compilation failures, execution speed, and remaining budget.

Concentrate the core implementation on harness generation and repair. Seed generation and automated crash triage are optional extensions.

Compare conventional fuzzing, a fixed amount of AI assistance followed by equal allocation of fuzzing time, and the adaptive controller. Use the same CPU and inference-budget ceilings and report actual consumption separately. Account for generation, compilation, and failed attempts, rather than counting only successful fuzzing time.

Evaluation and deliverables

Measure target-library coverage over time and at the budget limit, valid harnesses per inference budget, time spent on invalid candidates, and reproducible deduplicated bugs where available. Use common instrumentation and exclude generated harness code from coverage comparisons. Preserve a consistent target-library coverage denominator across policies.

Repeat experiments with multiple seeds and report variability. Known-bug revisions may provide an additional reproducible check of bug-finding behavior. Finding a previously unknown vulnerability is a stretch outcome, not a requirement.

Submit the controller, harnesses, experiment runner, coverage and cost measurements, reproducible failure inputs where applicable, and a report explaining the allocation decisions.

Detailed assessment criteria

Strong submissions demonstrate valid harnesses, fair end-to-end resource accounting, repeated controlled comparisons, and an explanation of when AI assistance pays off. More generated code or raw crash files alone do not establish a better fuzzer.

Starting resources

- OSS-Fuzz-Gen
- OSS-Fuzz documentation
- LLVM libFuzzer documentation

Option 7: Cybersecurity Challenge Agents

Option 7: Resource-Bounded Cybersecurity Challenge Agents

Project description

Cybersecurity capture-the-flag challenges provide concrete environments in which an agent must inspect artifacts, use tools, and adapt its strategy to feedback. This project studies how execution support helps a local model solve such tasks within a limited budget.

The core question: Which agent execution mechanisms improve success and reduce wasted work on controlled cybersecurity challenges?

Hardware and software environment

Use an instructor-approved, piloted subset of containerized challenges with explicit success checks. Run challenge services and tools on CPU, with a local model on an edge machine. Keep the agent inside the challenge environment, with access only to assigned targets and artifacts. Scoring metadata, reference solutions, and answer keys must not be accessible to the agent.

Begin with a small subset the baseline model can make meaningful progress on. Completing every benchmark category or solving previously unknown vulnerabilities is outside the core requirement.

Phase 1: Baseline agent and execution measurements

Implement a simple tool-using agent with a fixed model, task instructions, tool set, and step budget. Use separate development and held-out challenges from NYU CTF Bench or a comparable environment.

Measure solve rate, time and tokens per attempt, tool-call latency, invalid commands, repeated ineffective actions, context growth, and timeouts. Inspect failed traces to distinguish missing model knowledge, poor tool interfaces, lost intermediate findings, and unproductive execution loops.

Verify successes through the challenge's independent outcome check. Use fresh challenge state for each attempt, and confirm that success was not obtained by reading evaluation files or existing solutions.

Phase 2: Improve one execution mechanism

Choose a mechanism justified by Phase 1: structured and compact tool feedback, checkpoint/restart, or a memory of useful findings across attempts. For example, a restart mechanism might retain verified observations while discarding an unproductive action history.

Compare against the simple agent and fixed retries with the same model, tools, and total budget. Keep environment state and agent memory consistent when restoring checkpoints. Evaluate the mechanism on held-out challenges and include an ablation that removes the proposed feature.

A multi-agent extension is optional, but must compare equal aggregate inference budgets rather than treating extra model calls as free.

Evaluation and deliverables

Report solved challenges, time and inference cost, repeated actions, tool errors, and performance by task category. Use repeated attempts where stochasticity matters, and show the full distribution of outcomes rather than only successful traces.

Submit the agent harness, execution mechanism, reproducible container setup, sanitized traces, independent scoring procedure, and failure analysis. Public challenge results measure benchmark performance; they do not establish general real-world attack capability.

Detailed assessment criteria

Strong submissions connect a measured execution bottleneck to a concrete mechanism, control model and budget differences, verify independent outcomes, and explain both improvements and remaining failures.

Starting resources

- NYU CTF Bench
- EnIGMA: Enhanced Interactive Generative Model Agent for CTF Challenges

Useful Resources

Useful Resources

Local inference and hardware

- llama.cpp: a candidate local inference runtime; verify support and performance for the actual model and device.
- OpenVINO GenAI NPU documentation: a starting point for supported Intel NPU experiments.
- AMD ROCm compatibility matrix: check the installed operating system, driver, hardware, and runtime combination.
- GMKtec EVO-X2 and ASUS NUC 16 Pro: product-level information; course-machine configuration will be confirmed separately.

Agent security and evaluation

- AgentDojo: prompt-injection evaluation for tool-using agents.
- TAMAS: adversarial multi-agent scenarios.
- A2ASecBench: protocol-aware multi-agent security evaluation.
- NYU CTF Bench: containerized cybersecurity challenges for evaluating agents.

Fuzzing

- OSS-Fuzz-Gen: LLM-assisted fuzz-target generation and evaluation.
- OSS-Fuzz documentation: build, harness, and evaluation guidance.
- LLVM libFuzzer: coverage-guided fuzzing and execution controls.

Practical experiment guidance

Use a pinned course-tested version of the chosen stack. Start with a pilot before launching a large experiment matrix. Save workload definitions, seeds, configuration, logs, and analysis code together. Preserve failed outcomes and explain exclusions.

For AWS experiments, confirm that the service is eligible for the university credit program before provisioning it. Set a project budget and shut down unused resources. Shared local inference is the default alternative when paid API access is unavailable.

Project deadlines and group policies are listed in the Overview tab. Course-specific access instructions, software images, machine reservation procedures, AWS account instructions, and grading weights will be added when confirmed.